

Module 4 Implementation Note

Reduced Alfvén Continuum and Eigenmode Benchmark

Python/MATLAB architecture, algorithms, data products, tests, and reproducibility

Status. The first conventional solver milestone is implemented. Python execution and all repository tests pass in the build environment. MATLAB source and tests are complete but were not executed here because a MATLAB runtime is unavailable. The implementation is frozen as a reduced verification benchmark; no PINN is included in this milestone.

Contents

1	Implementation objectives	2
2	Repository paths	2
3	Configuration schema	3
4	Python design	3
4.1	Data classes	3
4.2	Numerical pipeline	3
4.3	Public Module 4 interface	3
5	MATLAB design	4
6	Finite-volume matrix construction	4
7	Eigenvector conventions	4
8	Output definitions	5
9	Automated tests	5
9.1	Python tests executed in this build	5
9.2	MATLAB tests	5

10 Running the benchmark	6
11 Failure behavior	6
12 Reproducibility and limitations	6
13 Next implementation milestone	7

1 Implementation objectives

The implementation has six requirements:

1. read the same physical case in Python and MATLAB;
2. assemble the same self-adjoint two-harmonic operator independently;
3. produce ordered, normalized, sign-canonical eigenvectors;
4. expose language-neutral output files;
5. verify dimensions, symmetry, scaling, residuals, and grid convergence;
6. preserve the existing Module 4 scaffold interface used by the repository backbone.

2 Repository paths

Path	Role
<code>python/cases/reduced_ae_benchmark/case.json</code>	Python copy of the shared benchmark configuration.
<code>matlab/cases/reduced_ae_benchmark/case.json</code>	Byte-identical MATLAB copy.
<code>python/.../module4_alfven_continuum_eigenmodes/model.py</code>	Typed configuration and result containers.
<code>python/.../module4_alfven_continuum_eigenmodes/benchmark.py</code>	Profiles, matrix assembly, eigensolve, convergence, and output writer.
<code>python/.../module4_alfven_continuum_eigenmodes/solver.py</code>	Public module interface and backbone compatibility.
<code>matlab/src/+kinetic_mhd_pinn/+module4_.../</code>	MATLAB package containing the equivalent numerical stages.

<code>data/reference/reduced_ae_benchmark/</code>	Frozen Python reference outputs and figures.
<code>python/tests/test_module4_reduced_ae.py</code>	Python physics, regression, and output-contract tests.
<code>matlab/tests/test_module4_reduced_ae.m</code>	MATLAB physics and Python-reference comparison tests.

3 Configuration schema

The JSON case contains five active groups: **geometry**, **profiles**, and **mode**, together with **numerics** and **verification**. Both implementations parse SI quantities without implicit unit conversion. The two case files are tested for byte equality to prevent silent language divergence.

The duplicated JSON files are intentional because each language tree remains independently runnable. A test treats either file changing alone as an error.

4 Python design

4.1 Data classes

`model.py` defines immutable configuration dataclasses and result containers. Its `with_updates` helper creates controlled parameter variants for scaling and convergence tests without mutating the baseline case.

4.2 Numerical pipeline

The Python call sequence is

$$\text{load_case} \rightarrow \text{build_profiles} \rightarrow \text{assemble_operator} \rightarrow \text{solve_benchmark} \rightarrow \text{write_outputs}. \quad (4.1)$$

The sparse eigensolve uses `scipy.sparse.linalg.eigsh` with the smallest-magnitude eigenpairs, a tolerance of 10^{-12} , and a high iteration ceiling. The operator is positive for the baseline, so smallest magnitude and smallest algebraic eigenvalues coincide.

4.3 Public Module 4 interface

`AlfvenContinuumEigenmodesModule.run` retains the original architecture contract. The static method `solve_case` invokes the actual benchmark. This avoids breaking early backbone tests while

replacing the “scaffold only” status with a real, explicitly bounded capability.

5 MATLAB design

The MATLAB package follows the same numerical stages using package-qualified functions. `solver.m` has two behaviors: with no input it returns the original `ScaffoldModule`; with a path or decoded structure it runs `solve_benchmark`. The eigensolve uses `eigs` with symmetric/real flags and the lowest real eigenpairs.

MATLAB output names and columns match Python. The MATLAB case driver writes to `matlab/outputs/reduced_ae_benchmark`; it does not overwrite the frozen Python reference.

6 Finite-volume matrix construction

The coupling is scaled as $C = \epsilon_c \omega_m \omega_{m+1}$ times a Gaussian envelope, with $0 \leq \epsilon_c < 1$ enforced in both languages so the local two-harmonic matrix remains positive definite. The scalar radial matrix is assembled explicitly from face coefficients. The left boundary contributes no flux. The right Dirichlet boundary contributes $2D(1)/h^2$ to the final diagonal because the face is half a cell from the last unknown. The two-harmonic block operator is assembled from two copies of this radial matrix, diagonal continuum terms, and a diagonal coupling block.

The implementation deliberately uses the ordinary identity mass matrix. If later extensions use nonuniform quadrature, metric Jacobians, or finite elements, the problem should be generalized to

$$\mathbf{A}\mathbf{x} = \omega^2 \mathbf{B}\mathbf{x} \tag{6.1}$$

with a positive-definite mass matrix $\mathbf{B}$.

7 Eigenvector conventions

Raw eigenvectors from sparse solvers can differ by sign. Each vector is rescaled so that

$$h\mathbf{x}^\top \mathbf{x} = 1, \tag{7.1}$$

and then flipped if its largest-magnitude component is negative. This makes CSV regression and Python/MATLAB comparisons reproducible while respecting the physically arbitrary global sign.

For future degenerate or nearly degenerate eigenspaces, individual eigenvector comparison will be insufficient. Subspace angles or projector differences should then replace component correlations.

8 Output definitions

All numerical tables use comma-separated text to support MATLAB, Python, and external inspection. Frequencies are written in both angular units and hertz. Metadata records the exact normalization and boundary conditions. Figures are secondary products; CSV and JSON files are authoritative.

9 Automated tests

9.1 Python tests executed in this build

The test suite verifies:

1. byte-identical Python/MATLAB case files;
2. positive density and Alfvén speed;
3. analytic crossing inside the domain and a nonzero avoided crossing;
4. matrix symmetry and positive computed eigenvalues;
5. eigenvector normalization and small algebraic residuals;
6. exact B_0 and density scaling;
7. zero off-diagonal blocks and ordered uncoupled branches when coupling is disabled;
8. monotonic grid-refinement improvement;
9. regression against frozen eigenfrequencies;
10. completeness of the language-neutral output contract;
11. preservation of the original repository architecture tests.

9.2 MATLAB tests

The MATLAB suite repeats symmetry, residual, normalization, scaling, convergence, and reference-frequency tests. It can be executed with

```
cd matlab
addpath('src')
results = runtests('tests');
assert(all([results.Passed]));
```

The repository test script runs MATLAB automatically when the executable is on the system path.

10 Running the benchmark

From the repository root, the Python reference case is regenerated with

```
python python/cases/reduced_ae_benchmark/run_case.py --doc-figures
```

A user-selected output directory can be supplied with `-output`. The MATLAB case is run with

```
cd matlab
addpath('src')
addpath('cases/reduced_ae_benchmark')
run_case
```

After MATLAB outputs exist, cross-language comparison is run with

```
python python/cases/reduced_ae_benchmark/compare_python_matlab.py \
    data/reference/reduced_ae_benchmark \
    matlab/outputs/reduced_ae_benchmark
```

The comparator checks eigenfrequency differences and sign-invariant component correlations.

11 Failure behavior

The solvers fail explicitly for nonpositive field, radius, or density; invalid edge-density fraction; nonadjacent harmonics in the baseline model; too few radial cells; invalid eigenpair count; zero eigenvectors; or nonpositive computed ω^2 . These checks prevent silently generating meaningless reference data.

12 Reproducibility and limitations

The reference CSV files are generated by Python in the current environment. MATLAB parity is specified and testable but not claimed as executed. Sparse eigensolvers can show small platform-dependent variation, so cross-language tolerance is looser than same-language regression tolerance.

No machine-learning code is part of this update. Module 9 will later consume this benchmark as a trusted target for an eigenvalue PINN. The conventional solver, case JSON, and reference outputs should be versioned before training data or surrogate results are added.

13 Next implementation milestone

The next narrowly defined milestone should implement an eigenvalue PINN for the coupled radial-envelope equations in Section 7 of the physics note. Its acceptance criteria should include frequency error, eigenfunction error up to sign, differential residual, boundary residual, normalization, orthogonality, and robustness across parameter scans. The PINN must not replace the matrix solver as the reference implementation.